Technical Report: Event Prediction in a Controlled Traffic Simulation

March 29, 2026

Contents

Contents

1 Scope **2**

2 Application and Simulation Design **2**

3 Model Families **5**

3.1 Empirical and Mechanistic Baselines 5

3.2 Learned Temporal Models 5

4 Learning Objectives **9**

4.1 Condition Calibration 9

5 Evaluation **11**

6 Results **14**

6.1 Primary Benchmark: Adaptive 60/20 14

6.2 Fixed-Horizon Condition Forecasting 16

6.3 Baseline Simulator vs Richer Simulator at the Same Scale 17

6.4 Fixed vs Adaptive at the Same Scale 19

7 Interpretation **21**

8 Limitations and Next Steps **22**

9 Conclusion **23**

1 Scope

This report summarizes the current benchmark for event prediction in a signalized traffic simulator and is written to remain extensible as the repository grows. The current application is a four-way traffic intersection with both `adaptive` and `fixed` signal-control modes. The benchmark now evaluates:

- next-event prediction
- short rollout forecasting
- fixed-horizon condition forecasting

The core prediction problem is

$$p(t_{i+1}, m_{i+1} \mid \mathcal{H}_i, s_i),$$

where t_{i+1} is the next event time, m_{i+1} is the next event mark, $\mathcal{H}_i$ is the event history, and s_i is the explicit system state.

2 Application and Simulation Design

The current environment is a single signalized intersection with four inbound approaches: `north`, `south`, `east`, and `west`. The emitted event families are:

- `phase_change`
- `vehicle_arrival`
- `vehicle_departure`

The simulator is implemented as a discrete-event system. At any simulation time t, the environment state contains:

$$s(t) = \big(q_N(t), q_S(t), q_E(t), q_W(t), \rho(t), \tau_\rho(t), w_N(t), w_S(t), w_E(t), w_W(t)\big),$$

where $q_\ell(t)$ is the queue length on lane ℓ, $\rho(t)$ is the active signal phase, $\tau_\rho(t)$ is elapsed time in the active phase, and $w_\ell(t)$ summarizes waiting-time information for that lane. The total queue and corridor pressures are

$$Q_{\text{tot}}(t) = \sum_{\ell \in \{N,S,E,W\}} q_\ell(t), \qquad Q_{\text{NS}}(t) = q_N(t) + q_S(t), \qquad Q_{\text{EW}}(t) = q_E(t) + q_W(t).$$

The simulation maintains an event calendar $\mathcal{E}$ ordered by event time. Each step removes the earliest event

$$e^\star = \arg\min_{e \in \mathcal{E}} t(e),$$

advances the global clock to $t(e^\star)$, updates state, records the event, and schedules any downstream events implied by the new state. Operationally, this means the environment is not stepped on a fixed time grid; it advances directly from one arrival, departure, or phase transition to the next.

Arrival processes are stochastic and lane-specific. If λ_ℓ is the nominal arrival rate for lane ℓ, then arrivals are sampled through an exponential inter-arrival process,

$$\Delta t_\ell^{\text{arr}} \sim \text{Exp}(\lambda_\ell),$$

with the sampled arrival inserted into the event calendar. When an arrival occurs, the corresponding queue is incremented,

$$q_\ell(t^+) = q_\ell(t^-) + 1.$$

Departures are service events that depend on both queue occupancy and phase compatibility. If lane ℓ is currently served by phase $\rho(t)$ and $q_\ell(t) > 0$, then departures are scheduled using a service gap g_ℓ, so that

$$t^{\text{next}}_{\text{dep},\ell} = t + g_\ell.$$

When a departure fires,

$$q_\ell(t^+) = \max(0, q_\ell(t^-) - 1).$$

This event logic creates the basic queueing dynamics that the prediction models must learn.

Mathematics of the Richer Simulator Profile

The richer simulator keeps the same event-driven state-transition structure, but changes the stochastic primitives that generate arrivals and departures. In the earlier baseline profile, each lane had a stationary Poisson arrival process with constant rate λ_ℓ and a shared service headway $g_\ell \equiv g$. In the richer profile, both of these simplifications are relaxed.

Time-varying corridor demand. Each lane ℓ has a nominal base rate $\bar{\lambda}_\ell$, but the instantaneous arrival rate becomes a function of time:

$$\lambda_\ell(t) = \bar{\lambda}_\ell \, w(t) \, b_\ell(t),$$

where $w(t)$ is a smooth demand wave shared across the whole intersection and $b_\ell(t)$ is a corridor-specific bias term. The current implementation uses

$$w(t) = 1 + 0.22 \sin\left(2\pi \frac{t}{T}\right),$$

where T is the simulation horizon. This introduces a low-frequency demand oscillation over the run.

The corridor bias is piecewise in time. For north-south lanes,

$$b_\ell(t) = \begin{cases} 1.18, & 0.18T \le t \le 0.48T, \\ 0.92, & \text{otherwise}, \end{cases} \qquad \ell \in \{N, S\},$$

and for east-west lanes,

$$b_\ell(t) = \begin{cases} 1.16, & 0.52T \le t \le 0.82T, \\ 0.90, & \text{otherwise}, \end{cases} \qquad \ell \in \{E, W\}.$$

This creates corridor-specific demand pulses, so the dominant pressure direction changes over the course of a run.

Bursty arrivals. Conditional on the instantaneous rate $\lambda_\ell(t)$, the baseline gap is still exponential:

$$\Delta \tilde{t}^{\text{arr}}_\ell \sim \text{Exp}(\lambda_\ell(t)).$$

However, the richer profile applies a burst mechanism. Let $B_\ell \sim \text{Bernoulli}(p_\ell)$, where p_ℓ is the burst probability for lane ℓ. Then the realized inter-arrival gap is

$$\Delta t_\ell^{\text{arr}} = \begin{cases} \max(\delta_{\min}, \alpha_\ell \Delta \tilde{t}_\ell^{\text{arr}}), & B_\ell = 1, \\ \Delta \tilde{t}_\ell^{\text{arr}}, & B_\ell = 0, \end{cases}$$

where $\alpha_\ell \in (0, 1)$ is a lane-specific burst scale factor and $\delta_{\min} = 0.2$ seconds prevents unrealistically small gaps. This produces short local platoons without requiring a full car-following model.

Lane-specific service dynamics. Departures in the richer simulator are also heterogeneous. Each lane has a nominal service headway $\bar{g}_\ell$ and a turning-share parameter θ_ℓ. The realized service headway at time t is

$$g_\ell(t) = \max\left(g_{\min}, \bar{g}_\ell + 0.22 \min\left(4, \frac{q_\ell(t)}{8}\right) + 0.7\,\theta_\ell + \beta_\ell(t)\right),$$

where $g_{\min} = 1.4$ seconds and $\beta_\ell(t)$ is a phase-compatibility bonus:

$$\beta_\ell(t) = \begin{cases} -0.08, & \text{if lane } \ell \text{ is currently being served,} \\ 0, & \text{otherwise.} \end{cases}$$

The queue-dependent term makes long queues discharge more slowly than in the constant-headway baseline, the turning-share term penalizes lanes with more turning behavior, and the negative phase bonus slightly rewards active service.

The richer profile therefore changes the generative process from

$$(\lambda_\ell, g_\ell) = (\text{constant}, \text{constant})$$

to

$$(\lambda_\ell(t), g_\ell(t)) = (\text{time-varying, bursty, lane-specific, state-dependent}).$$

That change matters because it removes a large amount of regularity from the synthetic environment. Exact next-event prediction becomes harder: the model must now infer whether an event stream is entering a burst regime, whether a corridor pulse is underway, and whether a particular lane is likely to clear more slowly than another despite having the same signal state.

The richer simulator is still not a full microscopic traffic model, but mathematically it is a substantial step away from a homogeneous queueing benchmark. It introduces nonstationarity, local clustering, and lane heterogeneity while preserving an interpretable closed-form event generator.

The simulator tracks queue state, phase state, departures, and waiting times. It supports two control regimes:

- **Fixed control:** green durations remain at the base schedule.
- **Adaptive control:** green durations respond to queue pressure and arrival-rate pressure.

The signal cycle alternates between north-south service and east-west service with intermediate clearance phases. If d_{NS} and d_{EW} denote the active green durations, then the nominal phase order is

$$\texttt{NS_GREEN} \to \texttt{ALL_RED} \to \texttt{EW_GREEN} \to \texttt{ALL_RED}.$$

Under fixed control, d_{NS} and d_{EW} are constants. Under adaptive control they become state-dependent functions.

For adaptive control, the green duration is computed as

$$d_{\text{green}} = \text{clip}\left(d_{\text{base}} + w_q(q - q_0) + w_\lambda(\lambda - \lambda_0), d_{\min}, d_{\max}\right),$$

where q is corridor queue pressure and λ is corridor arrival-rate pressure.

In practice, q is a corridor aggregate such as Q_{NS} or Q_{EW}, and λ is estimated from recent lane-arrival observations rather than treated as a fully observed oracle. The clipping operator enforces realistic lower and upper bounds, preventing the controller from collapsing into excessively short or excessively long greens.

The simulator emits a full event trace

$$\{(t_i, m_i, a_i)\}_{i=1}^{T},$$

where a_i contains event attributes such as lane, phase, queue-after-update, and any controller metadata. These traces support both event-level supervision and the derived condition targets used later in evaluation.

This simulation design is intentionally stylized rather than microscopically realistic. It omits turning movements, multi-lane conflicts, pedestrian phases, and spillback into upstream links. Its purpose is to provide a controlled event-driven benchmark in which queue dynamics, controller logic, and stochastic arrivals are all present and measurable.

This environment is a good benchmark because it combines explicit state, stochastic arrivals, queueing, and controller logic while still remaining interpretable.

3 Model Families

3.1 Empirical and Mechanistic Baselines

Global rate baseline. Predict the next event from global recurrence statistics. For each label m, estimate an average recurrence gap $\bar{\Delta}_m$ and predict

$$\hat{t}_m = t + \max(\epsilon, \bar{\Delta}_m - (t - t_m^{\text{last}})).$$

Transition baseline. Predict the next event from empirical transition probabilities and average transition delay:

$$\hat{m}_{i+1} = \arg\max_m \hat{p}(m \mid m_i), \qquad \hat{t}_{i+1} = t_i + \mathbb{E}[\Delta t \mid m_i, m_{i+1}].$$

Mechanistic baseline. Predict the earliest plausible next event by constructing candidates for next phase change, next lane arrival, and next lane departure, then selecting

$$(\hat{t}, \hat{m}) = \arg\min_{(t,m)\in\mathcal{C}} t.$$

3.2 Learned Temporal Models

All learned models predict next-event type and next-event delay. Some also predict fixed-horizon traffic conditions directly.

For a generic event sequence, let the i-th observation be (t_i, m_i, s_i), where t_i is the event time, m_i is the event mark, and $s_i \in \mathbb{R}^{d_s}$ is the explicit simulator state. Define the inter-event gap

$$\Delta t_i = t_i - t_{i-1},$$

an event embedding $e_i = E[m_i]$, and a continuous-time feature map $\phi(\Delta t_i)$. The basic per-step input used by the learned models is

$$x_i = [\, e_i \,\|\, \phi(\Delta t_i) \,].$$

After temporal encoding, the latent history vector is fused with explicit state using

$$u_i = \psi([\, h_i \,\|\, s_i \,]),$$

where h_i is the history representation and $\psi(\cdot)$ is an MLP-style projection. Event-type probabilities are then computed as

$$\hat{p}(m_{i+1} \mid \mathcal{H}_i, s_i) = \mathrm{softmax}(W_m u_i + b_m),$$

while the next-event delay is predicted through

$$\hat{\Delta} t_{i+1} = \exp(W_t u_i + b_t).$$

GRU-based neural TPP.

$$h_i = \mathrm{GRU}(h_{i-1}, x_i), \qquad \hat{p}(m_{i+1} \mid h_i, s_i), \qquad \hat{\Delta} t_{i+1} = f(h_i, s_i).$$

More explicitly, the recurrent update can be written in the standard gated form

$$r_i = \sigma(W_r x_i + U_r h_{i-1} + b_r), \qquad z_i = \sigma(W_z x_i + U_z h_{i-1} + b_z),$$

$$\tilde{h}_i = \tanh(W_h x_i + U_h (r_i \odot h_{i-1}) + b_h),$$

$$h_i = (1 - z_i) \odot h_{i-1} + z_i \odot \tilde{h}_i.$$

The hidden state h_i summarizes recent event order and timing, while s_i injects the current physical state of the intersection. The fused representation is

$$u_i = \psi([\, h_i \,\|\, s_i \,]),$$

which is then consumed by the event-type and event-time heads.

Operationally, this model builds a sliding context of recent event labels and inter-event times. Each event is embedded, concatenated with timing information, and passed through a GRU. The final hidden state is then fused with explicit simulation state features such as queue lengths, current phase, elapsed phase time, and lane-level departure status.

This architecture is the simplest learned event model in the repository. Its strengths are:

- low implementation complexity
- strong one-step event prediction
- a clear division between temporal history encoding and explicit state conditioning

Its main limitation is that it has no direct condition-forecasting head. When asked about congestion or corridor pressure, it must roll itself forward event by event and derive those conditions indirectly from the synthetic future trajectory.

Multitask GRU TPP. Use the same recurrent backbone but add fixed-horizon condition heads:

$$\hat{c}_{i,h} = g_h(h_i, s_i), \qquad h \in \{10s, 30s, 60s\}.$$

In practice, the model shares the recurrent state h_i and fused representation u_i across several outputs. For each horizon h and each binary condition k, it predicts

$$\hat{p}(c_{i,h,k} = 1 \mid \mathcal{H}_i, s_i) = \sigma(w_{h,k}^{\top} u_i + b_{h,k}),$$

so that the condition vector for horizon h is

$$\hat{\mathbf{c}}_{i,h} = \sigma(W_h^{(c)} u_i + b_h^{(c)}).$$

This creates a shared-latent multitask system in which the same temporal representation must support both event-level forecasting and fixed-time condition forecasting.

Architecturally, this model begins with the same recurrent event-history encoder as the plain GRU TPP. The difference is that the shared hidden representation is used by multiple heads:

- an event-type head
- an event-time head
- direct condition heads for the fixed horizons $10s$, $30s$, and $60s$

That multitask design matters because it changes the supervision signal. The model is not only encouraged to predict the very next event; it is also encouraged to learn latent features that are predictive of medium-range traffic conditions. In practice this has made it the strongest overall condition forecaster in the larger adaptive benchmark.

Another important detail is that the direct condition outputs are calibrated post hoc. So the architectural path is:

1. recurrent temporal encoding
2. fusion with explicit state
3. direct probabilistic condition outputs
4. post-hoc calibration and thresholding

This is the current best example in the repo of a model whose architecture is aligned with the condition-forecast task rather than relying only on event rollout.

Continuous-time LSTM-style TPP. Decay the hidden state between events:

$$h_i^- = g(\Delta t_i) \odot h_{i-1}, \qquad h_i = \mathrm{LSTM}(h_i^-, x_i).$$

This gives the model an explicit continuous-time inductive bias.

A more detailed view separates the cell state c_i and hidden state h_i. If $d_i = g(\Delta t_i)$ is a learned decay gate, then one simple continuous-time update is

$$c_{i-1}^- = d_i \odot c_{i-1}, \qquad h_{i-1}^- = \tanh(c_{i-1}^-) \odot o_{i-1},$$

followed by the usual LSTM transition

$$f_i = \sigma(W_f x_i + U_f h_{i-1}^- + b_f), \qquad \iota_i = \sigma(W_\iota x_i + U_\iota h_{i-1}^- + b_\iota),$$

$$\tilde{c}_i = \tanh(W_c x_i + U_c h_{i-1}^- + b_c),$$

$$c_i = f_i \odot c_{i-1}^- + \iota_i \odot \tilde{c}_i, \qquad o_i = \sigma(W_o x_i + U_o h_{i-1}^- + b_o),$$

$$h_i = o_i \odot \tanh(c_i).$$

The resulting h_i is fused with s_i in the same way as the recurrent GRU variants, and the model emits both event predictions and calibrated condition probabilities.

The key architectural idea here is that the hidden state evolves not only when an event arrives, but also as elapsed time passes. The model therefore represents irregular event spacing more directly than a standard recurrent network that only updates on discrete steps.

In implementation terms, the model:

- embeds recent event labels
- applies an elapsed-time transformation before the recurrent update
- combines the resulting hidden state with explicit traffic-state features
- predicts both event outputs and fixed-horizon condition probabilities

This architecture is attractive because the traffic simulator is genuinely asynchronous. Event gaps are not uniform, and those gaps matter. The continuous-time inductive bias has been especially helpful for time MAE, where this model remains one of the strongest performers.

Relative to the multitask GRU, the continuous-time LSTM is less explicitly task-specialized around condition forecasting, but more specialized around irregular timing dynamics. That is why it tends to be especially strong on timing metrics while remaining competitive on direct condition prediction.

Transformer TPP.

$$z_{1:i} = \text{Transformer}(x_{1:i}),$$

then predict the event and fixed-horizon conditions from the pooled causal representation.

Let $X_i = [x_1, \ldots, x_i]^\top$. The embedded input sequence is

$$H^{(0)} = X_i W_{\text{in}} + P_i,$$

where P_i contains positional and temporal encodings. Each transformer block applies masked self-attention

$$Q = H^{(\ell)} W_Q, \qquad K = H^{(\ell)} W_K, \qquad V = H^{(\ell)} W_V,$$

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right),$$

$$\tilde{H}^{(\ell)} = AV,$$

where M is a causal mask that prevents access to future events. After residual and feed-forward updates across L layers, the model extracts a pooled representation z_i from the final causal context:

$$z_i = \text{Pool}(H_{1:i}^{(L)}).$$

The fused prediction state is then

$$u_i = \psi([\, z_i \,\|\, s_i \,]),$$

from which both event and condition heads are produced.

The transformer variant replaces recurrence with causal self-attention over the recent event sequence. Instead of compressing the full history step by step into a single recurrent state, it allows the model to revisit earlier events through attention weights. That is useful when the next event depends on patterns that span multiple earlier steps rather than only the latest hidden state.

In the current implementation, the architecture includes:

- event-label embeddings
- timing features
- positional or order information
- causal masking so future events are never exposed during prediction
- a pooled or prediction-token-style representation used for event and condition heads

The transformer also includes rollout-aware training, which is an architectural and optimization choice meant to reduce the gap between one-step supervision and autoregressive deployment. That matters because attention models can look good under teacher forcing but drift when their own predictions are fed back in.

Conceptually, the transformer is the model in the repository that is most willing to trade hand-crafted recurrence structure for broader context access. In the larger adaptive benchmark that tradeoff looks increasingly worthwhile: it is now competitive with the strongest direct-condition models rather than clearly trailing them.

4 Learning Objectives

The event objective is

$$\mathcal{L}_{\text{event}} = \mathcal{L}_{\text{type}} + \lambda \mathcal{L}_{\text{time}},$$

where $\mathcal{L}_{\text{type}}$ is cross-entropy on event labels and $\mathcal{L}_{\text{time}}$ is a robust loss on log-delay.

For the direct-condition models, the multitask objective is

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{event}} + \beta \mathcal{L}_{\text{cond}},$$

where $\mathcal{L}_{\text{cond}}$ is a fixed-horizon binary loss over condition labels such as congestion, severe queue, and corridor imbalance.

The current system also applies post-hoc Platt-style calibration per condition dimension, which is why the benchmark now reports probability-aware metrics such as Brier score and log loss.

4.1 Condition Calibration

Condition forecasting in this repository is probabilistic, not only binary. For a condition logit z, the raw model probability is

$$\tilde{p} = \sigma(z).$$

However, raw neural scores are often not well calibrated: a model may rank examples correctly while still being systematically overconfident or underconfident. To address that, the direct-condition models apply a post-hoc affine calibration per condition dimension,

$$\hat{p} = \sigma(az + b),$$

where a and b are fit on held-out logits and targets after the base model has been trained. This is a Platt-style calibration step applied separately for each condition output.

In the current implementation, calibration is performed independently for every horizon-condition pair. That means the system learns separate calibrators for, for example,

- `congested` at $10s$
- `congested` at $30s$
- `pressure_imbalance` at $60s$

rather than forcing a single shared probability transform across all condition types. This matters because different conditions have different prevalence, different difficulty, and different logit distributions.

After calibration, binary decisions are formed by thresholding the calibrated probabilities:

$$\hat{y} = \mathbf{1}\{\hat{p} \geq \tau\},$$

where τ is tuned per condition dimension. So there are two separate post-training steps:

1. calibrate the probability scale
2. choose the decision threshold used for balanced-accuracy-style evaluation

This distinction is important. Calibration answers the question "does a predicted probability of 0.8 actually correspond to roughly 80% empirical frequency?" Threshold tuning answers the different question "at what probability cutoff should the model declare that the condition is present?" A model can therefore have good ranking behavior but poor calibration, or good calibration but a suboptimal threshold.

Figure 1 shows the pooled reliability curves for the main learned condition models on the adaptive 60/20 benchmark. The dashed diagonal is perfect calibration. Curves close to the diagonal indicate that predicted probabilities match empirical frequencies, while curves far from the diagonal indicate miscalibration.

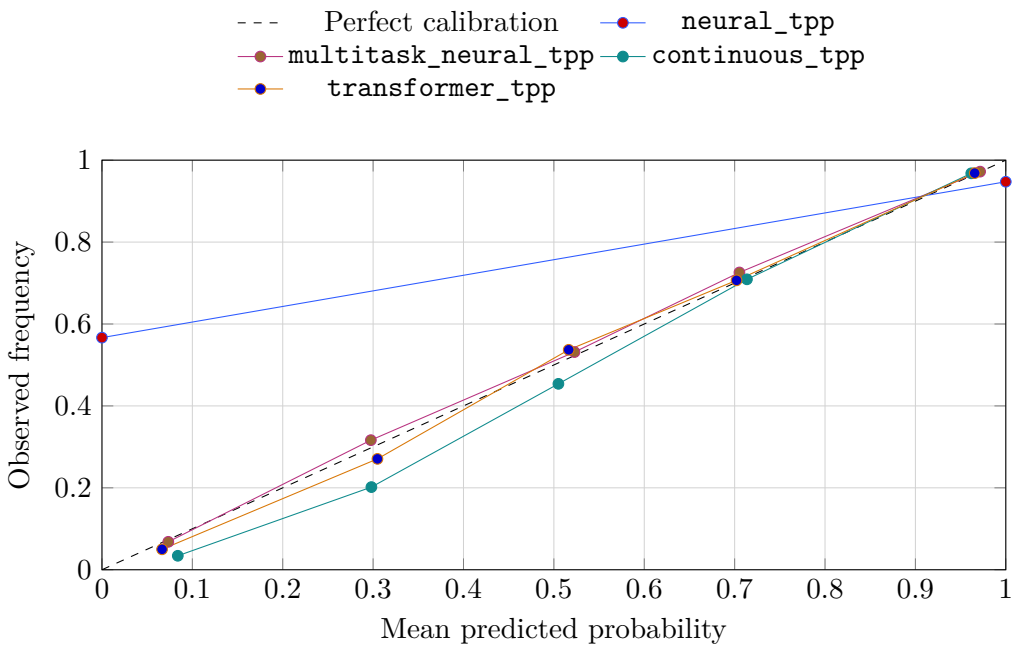

Figure 1: Pooled reliability curves for fixed-time condition forecasts on the adaptive 60/20 benchmark. Empty calibration bins are omitted. The direct-condition models lie much closer to the diagonal than the rollout-derived `neural_tpp`, whose condition probabilities are concentrated near 0 and 1.

5 Evaluation

The current benchmark evaluates:

1. one-step next-event prediction
2. short-horizon rollout forecasting
3. fixed-time condition forecasting at $10s$, $30s$, and $60s$

These three forecast tasks answer different questions.

One-step next-event forecasting. This is the most local prediction task. At context index i, the model is given the observed history $\mathcal{H}_i$ and explicit state s_i, and it must predict the very next discrete event:

$$(\hat{m}_{i+1}, \hat{t}_{i+1}).$$

The target is the immediate next realized event (m_{i+1}, t_{i+1}). This task measures whether the model understands the fine-grained event mechanics of the system: what kind of event fires next, and approximately when.

Short-horizon rollout forecasting. This is an autoregressive sequence forecast rather than a single-step forecast. Starting from $(\mathcal{H}_i, s_i)$, the model predicts the next event, updates a replay state with that prediction, then predicts again from its own synthetic future. If the rollout horizon is K, the predicted future is

$$\hat{\mathcal{R}}_{i,K} = \{(\hat{m}_{i+1}, \hat{t}_{i+1}), \ldots, (\hat{m}_{i+K}, \hat{t}_{i+K})\}.$$

This forecast is compared with the realized K-event future

$$\mathcal{R}_{i,K} = \{(m_{i+1}, t_{i+1}), \ldots, (m_{i+K}, t_{i+K})\}.$$

In the current benchmark, $K \in \{5, 10\}$. The rollout task is stricter than one-step prediction because errors compound. A model that is good at the first next event but poor at maintaining state consistency will degrade quickly under rollout.

Fixed-time condition forecasting. This task asks a higher-level operational question: what traffic condition will hold after a fixed amount of time has passed? For horizon $h \in \{10s, 30s, 60s\}$, the model predicts a vector of binary conditions

$$\hat{\mathbf{c}}_{i,h} = (\hat{c}_{i,h,1}, \ldots, \hat{c}_{i,h,K_c}),$$

where each dimension corresponds to a named traffic condition. The target is obtained by advancing either the real event trace or a model rollout to time $t_i + h$, reconstructing the future state $s(t_i + h)$, and evaluating the condition flags on that state.

This task is different in meaning from event forecasting. It does not primarily ask whether the model gets each intervening arrival or departure exactly right. Instead, it asks whether the model correctly anticipates a more general future regime of the intersection.

The condition definitions used in the current benchmark are thresholded functions of replay-state queues:

$$\text{congested}(s) = \mathbf{1}\{Q_{\text{tot}} \geq 12\},$$

$$\text{severe_queue}(s) = \mathbf{1}\{\max_{\ell} q_\ell \geq 6\},$$

$$\text{ns_pressure_high}(s) = \mathbf{1}\{Q_{\text{NS}} \geq 8\}, \qquad \text{ew_pressure_high}(s) = \mathbf{1}\{Q_{\text{EW}} \geq 8\},$$

$$\text{pressure_imbalance}(s) = \mathbf{1}\{|Q_{\text{NS}} - Q_{\text{EW}}| \geq 5\}.$$

These conditions should be interpreted as coarse traffic regimes rather than exact physical thresholds from a real deployment. They were chosen to create a meaningful distinction between light, moderate, and asymmetric demand states in the simulated environment.

The benchmark therefore spans three semantic levels:

- event-level forecasting: what happens next
- sequence-level forecasting: what chain of events unfolds next
- state/condition-level forecasting: what regime the system will be in after some horizon

Each level is useful for a different downstream purpose. Event forecasts are closest to process modeling, rollout forecasts test stability under autoregressive use, and condition forecasts are closest to operational decision support.

The one-step event metrics are defined as follows. Let N be the number of evaluated prediction points.

The main event metrics are:

- type accuracy
- family accuracy
- time MAE
- time RMSE

- within-2-seconds rate

Type accuracy measures exact event-label agreement:

$$\mathrm{Acc}_{\mathrm{type}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\hat{m}_{i+1} = m_{i+1}\}.$$

Family accuracy is coarser. If $\mathrm{fam}(m)$ maps an event label to one of {`phase_change`, `vehicle_arrival`, `vehicle_departure`}, then

$$\mathrm{Acc}_{\mathrm{family}} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\mathrm{fam}(\hat{m}_{i+1}) = \mathrm{fam}(m_{i+1})\}.$$

This metric is useful when the model gets the event family right even if it misses the exact subtype.

Let the timing residual be

$$\varepsilon_i = \hat{t}_{i+1} - t_{i+1}.$$

Then the absolute timing metrics are

$$\mathrm{MAE}_t = \frac{1}{N}\sum_{i=1}^{N} |\varepsilon_i|, \qquad \mathrm{RMSE}_t = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \varepsilon_i^2}.$$

The within-2-seconds rate is

$$\mathrm{Hit@2s} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{|\varepsilon_i| \le 2\}.$$

This last metric is operationally useful because many event forecasts can tolerate small timing error without changing the implied traffic interpretation.

The main condition metrics are:

- balanced accuracy
- Brier score
- log loss

For a binary condition $c \in \{0,1\}$, let $\hat{y}_i \in \{0,1\}$ be the thresholded prediction and $\hat{p}_i \in [0,1]$ the predicted probability. If TPR and TNR are the true positive rate and true negative rate, then balanced accuracy is

$$\mathrm{BalAcc} = \frac{1}{2}(\mathrm{TPR} + \mathrm{TNR}).$$

Balanced accuracy is preferred over raw accuracy because several traffic conditions, especially at longer horizons, are class-imbalanced and can otherwise be dominated by the majority class.

The Brier score measures squared probability error:

$$\mathrm{Brier} = \frac{1}{N}\sum_{i=1}^{N} (\hat{p}_i - c_i)^2.$$

Lower values indicate probabilities that are numerically close to the true outcomes.

The log loss is

$$\mathrm{LogLoss} = -\frac{1}{N}\sum_{i=1}^{N} \left[c_i \log \hat{p}_i + (1 - c_i)\log(1 - \hat{p}_i)\right].$$

This metric penalizes overconfident errors much more strongly than the Brier score, which makes it informative for decision-support use cases where poorly calibrated confidence is costly.

The benchmark also reports per-family and per-condition breakdowns. Those stratified views are important because a single pooled metric can hide qualitatively different behavior: a model may excel on departures but struggle on phase changes, or it may be well calibrated for congestion while remaining poorly calibrated for imbalance conditions.

Calibration is evaluated in two complementary ways. First, the benchmark reports Brier score and log loss, which summarize probabilistic quality numerically. Second, it computes pooled calibration bins over predicted condition probabilities. If a bin contains predictions in a score range $[u_j, u_{j+1})$, then the reported calibration point is

$$\left(\frac{1}{|B_j|} \sum_{i \in B_j} \hat{p}_i, \frac{1}{|B_j|} \sum_{i \in B_j} c_i \right).$$

Plotting these points against the diagonal $y = x$ yields a reliability curve. Curves near the diagonal indicate that predicted probabilities match observed frequencies, while curves above or below the diagonal indicate systematic underconfidence or overconfidence.

6 Results

6.1 Primary Benchmark: Adaptive 60/20

The strongest current benchmark is the larger adaptive run with:

- `train-runs = 60`
- `eval-runs = 20`

Figure 2 summarizes one-step results on this benchmark.

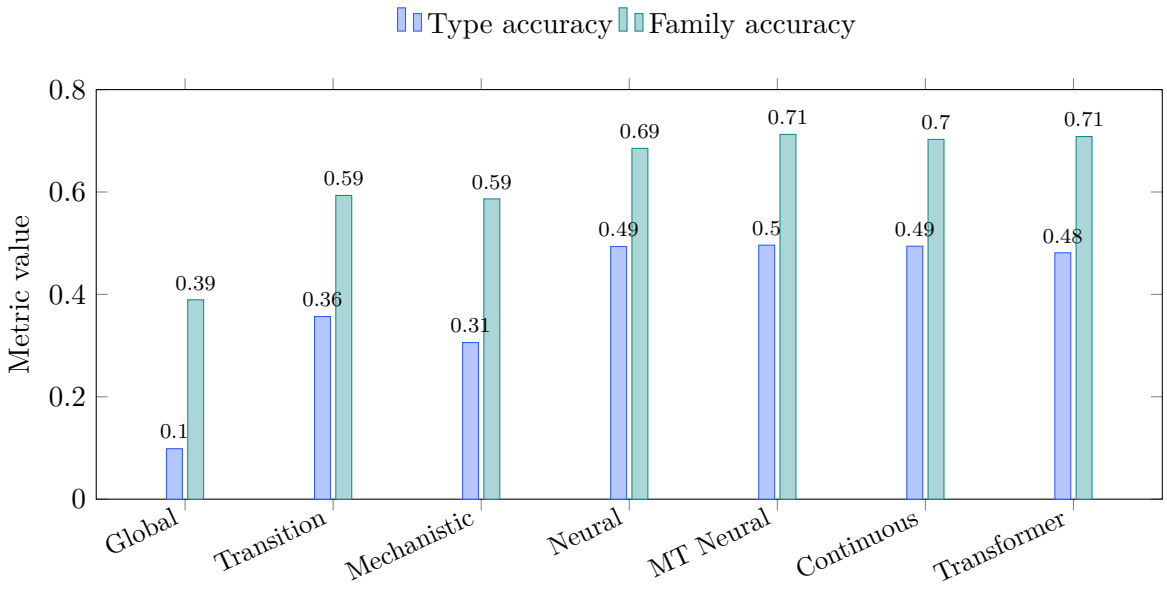

Figure 2: One-step event accuracy metrics on the adaptive 60/20 benchmark. Only accuracy-valued metrics are combined in this figure. The strongest learned models are tightly clustered, with `multitask_neural_tpp` slightly ahead on exact and family-level event identification.

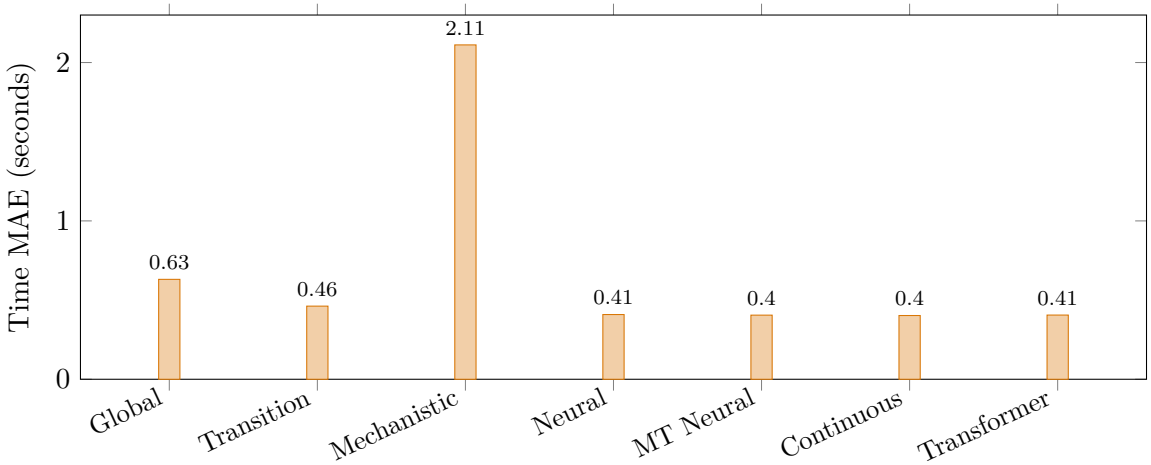

Figure 3: One-step timing error on the adaptive 60/20 benchmark. This figure is separated from the accuracy metrics because MAE is measured in seconds rather than normalized score units. `continuous_tpp` is best on timing, with the other learned models close behind.

The main one-step conclusions are:

- `multitask_neural_tpp` has the best exact type accuracy: 0.4962
- `multitask_neural_tpp` also has the best family accuracy: 0.7125
- `continuous_tpp` has the best time MAE: 0.4025

6.2 Fixed-Horizon Condition Forecasting

Figure 4 summarizes the fixed-horizon condition task on the same adaptive 60/20 benchmark.

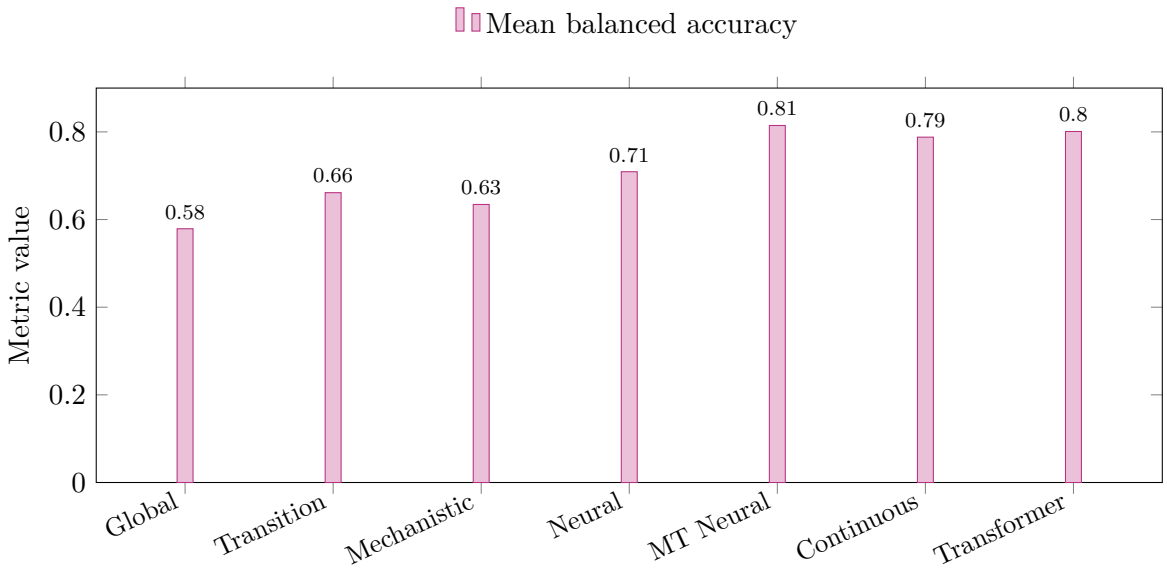

Figure 4: Fixed-horizon condition balanced accuracy averaged across all condition definitions and horizons on the adaptive 60/20 benchmark. This plot contains only normalized accuracy-style values. The direct-condition models are clearly separated from the rollout-derived condition forecasts.

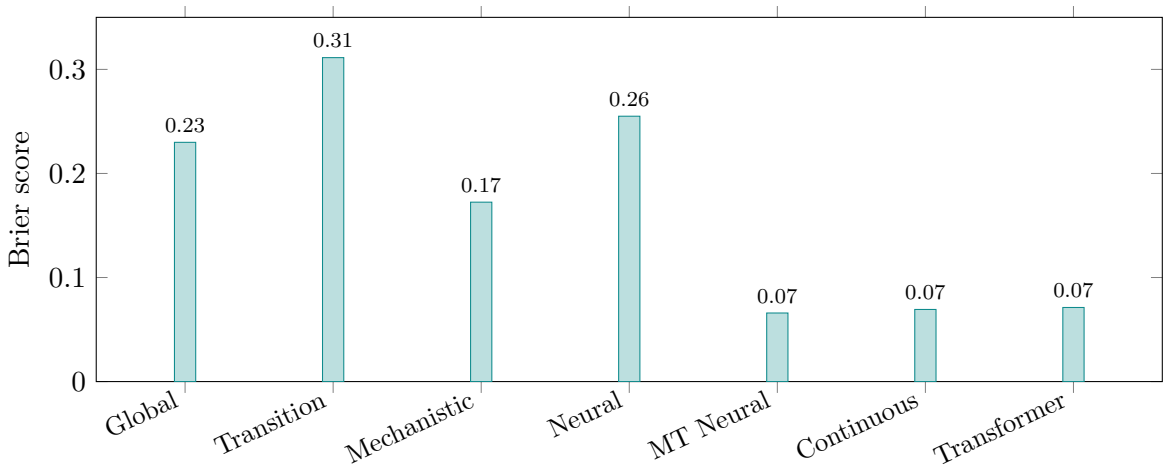

Figure 5: Fixed-horizon probabilistic quality measured by mean Brier score. Lower is better. `multitask_neural_tpp`, `continuous_tpp`, and `transformer_tpp` all produce much better calibrated condition probabilities than the rollout-derived baselines.

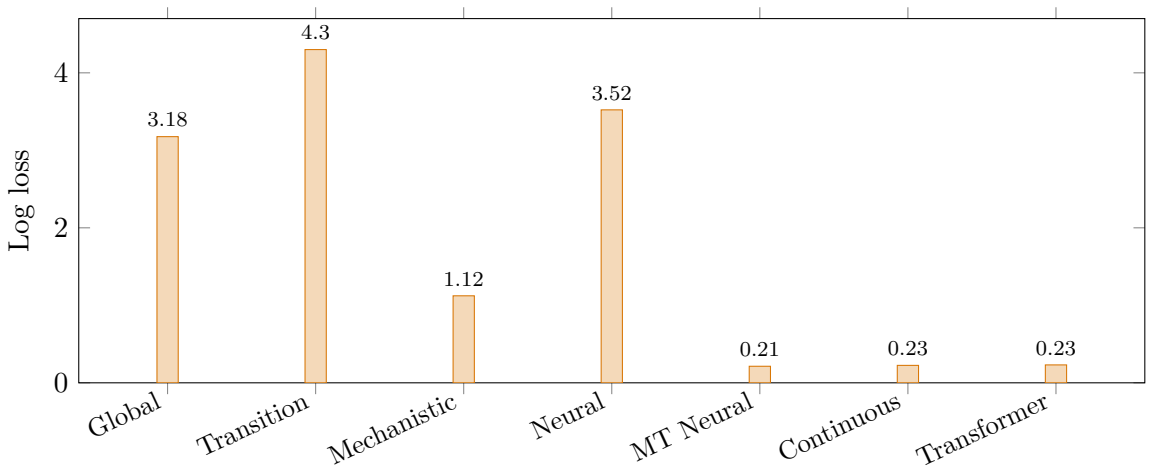

Figure 6: Fixed-horizon probabilistic quality measured by mean log loss. Lower is better. This view isolates overconfident errors and shows how strongly the direct-condition heads improve confidence quality.

The most important result in the report is that direct-condition models are now decisively stronger on condition forecasting:

Model	Mean Balanced Accuracy	Mean Brier	Mean Log Loss
`global_rate`	0.5790	0.2299	3.1767
`transition`	0.6613	0.3113	4.3002
`mechanistic`	0.6344	0.1724	1.1229
`neural_tpp`	0.7091	0.2550	3.5223
`multitask_neural_tpp`	**0.8146**	**0.0659**	**0.2136**
`continuous_tpp`	0.7880	0.0693	0.2250
`transformer_tpp`	0.8009	0.0712	0.2311

6.3 Baseline Simulator vs Richer Simulator at the Same Scale

The repository now also includes a matched comparison between the earlier baseline-style simulator and the newer richer simulator, both under adaptive control and both at the same 60/20 benchmark scale. This comparison isolates the effect of changing the traffic environment itself rather than changing the train/eval budget or controller regime.

The richer simulator differs from the baseline simulator in three main ways:

- arrivals are no longer stationary Poisson with a single fixed rate per lane; they include time-varying corridor pulses and bursty sub-platoons
- service is no longer governed by one shared saturation headway; headways now vary by lane
- turning-share penalties induce lane-specific discharge behavior, so not all queues clear equally quickly even when they have green

Figure 7 compares mean fixed-horizon condition balanced accuracy between the two simulator profiles. This figure uses only normalized balanced-accuracy values, so it stays on a single metric scale.

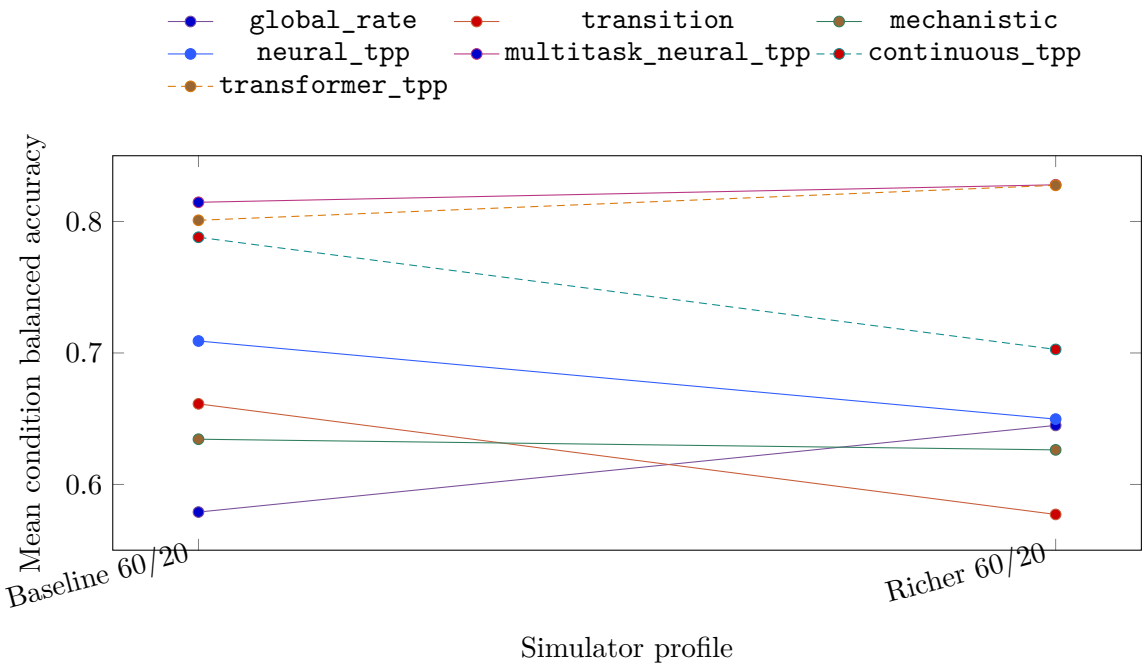

Figure 7: Mean fixed-time condition balanced accuracy under the earlier baseline simulator and the richer simulator, both at adaptive 60/20 scale. The richer simulator still makes exact event prediction harder, but after the latest direct-condition refinements it now exposes a stronger separation between the best calibrated condition models and the weaker empirical baselines.

Table 1 complements that condition-level view with one-step event metrics for representative models. Each cell reports `baseline/richer` at the same adaptive 60/20 scale.

Metric	Transition	MT Neural	Continuous	Transformer
Type accuracy	0.357 / 0.259	0.496 / 0.441	0.494 / 0.429	0.481 / 0.426
Family accuracy	0.593 / 0.491	0.713 / 0.682	0.703 / 0.693	0.708 / 0.648
Time MAE	0.462 / 0.472	0.405 / 0.403	0.403 / 0.402	0.405 / 0.411
Mean cond. bal. acc.	0.661 / 0.577	0.815 / 0.828	0.788 / 0.703	0.801 / 0.828

Table 1: Representative one-step and condition-summary results under the earlier baseline simulator and the richer simulator, both at adaptive 60/20 scale. Each cell is reported on a single metric scale and uses the same train/eval budget.

This baseline-versus-richer comparison changes the interpretation of the benchmark in an important way. The earlier simulator was still useful, but it was more regular: arrivals were simpler, effective service was more homogeneous, and the queue dynamics were easier to summarize with a small set of empirical averages. The richer simulator breaks that regularity. Under the richer profile, type accuracy falls for nearly every learned model, which indicates that exact event identity has become genuinely harder to infer from local history alone. At the same time, timing MAE changes very little for the strongest learned models, especially `continuous_tpp` and `multitask_neural_tpp`. That is a good sign: the richer simulator is not merely injecting arbitrary noise, but specifically making the mark-prediction problem more heterogeneous while leaving the temporal-scale modeling problem largely intact.

The condition metrics become more informative in the richer environment, but the most important update is that the latest direct-condition refinements now hold up under that harder setting. `continuous_tpp` still drops from 0.7880 to 0.7027, which confirms that richer heterogeneity exposes real weaknesses in that model's condition path. By contrast, the stronger direct-condition specialists now remain at or above their baseline-simulator condition scores: `multitask_neural_tpp` rises from 0.8146 to 0.8279, and the improved `transformer_tpp` rises from 0.8009 to 0.8275. That does not mean the richer simulator is easier overall; exact event typing still gets harder. Rather, it means the richer benchmark now rewards models that can fuse longer event context with explicit state-aware condition structure.

At the same time, the richer simulator sharpens the distinction between model families. The empirical `transition` baseline degrades strongly on both exact event prediction and condition balanced accuracy, which indicates that first-order transition statistics are no longer sufficient once arrivals become bursty and lane service becomes heterogeneous. The `mechanistic` baseline behaves differently: its exact type prediction degrades substantially, but its probabilistic condition metrics improve and its pressure-imbalance behavior becomes more structured. This is consistent with a hand-built queue model that still captures coarse directional pressure, but is less able to match the exact fine-grained event stream once lane-specific service rules matter more.

A final focused transformer pass further refined that story. Lowering the transformer's condition-loss weight from 0.10 to 0.08 preserved its richer-simulator condition advantage while recovering family accuracy in a focused rerun, reaching family accuracy 0.6759 with mean condition balanced accuracy 0.8285. That result is not yet folded into the all-model richer comparison table above, but it reinforces the same conclusion: on the richer simulator, the transformer is no longer a distant third condition model. It is effectively tied with `multitask_neural_tpp` when its condition path and loss balance are tuned to the task.

6.4 Fixed vs Adaptive at the Same Scale

The repository now includes a matching fixed-control benchmark at the same 60/20 scale, so the regime comparison can be made without mixing dataset sizes. Figure 8 compares mean fixed-time condition balanced accuracy under fixed and adaptive control only, which keeps the comparison on a single metric scale and a single train/eval configuration.

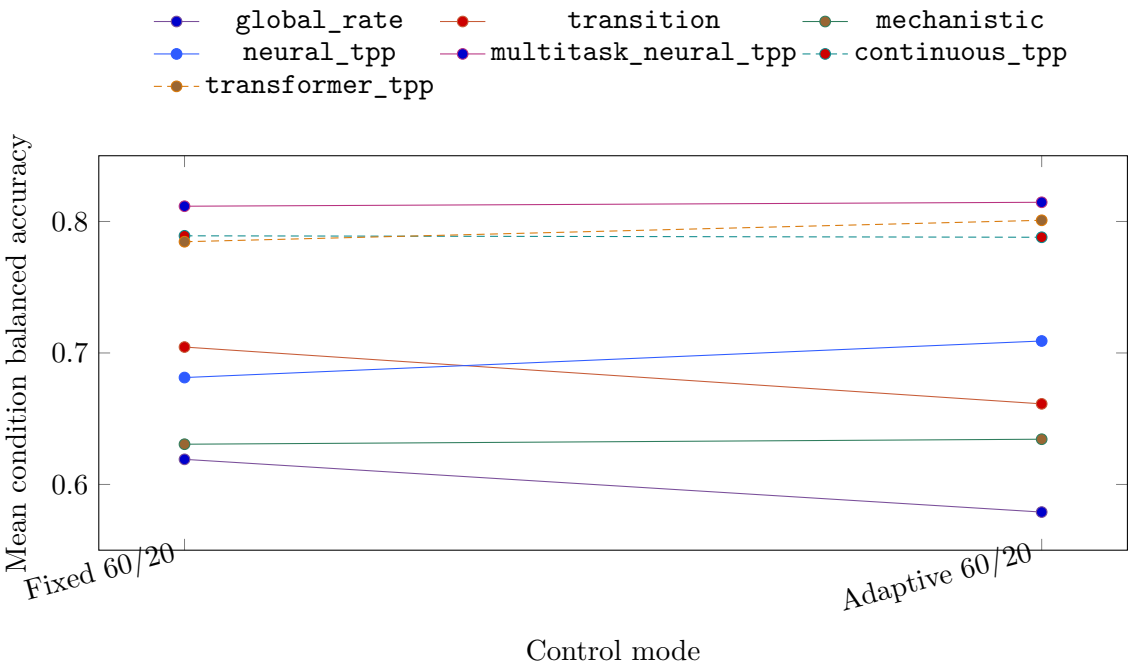

Figure 8: Mean fixed-time condition balanced accuracy under fixed and adaptive control for the same 60/20 benchmark scale. The strongest direct-condition models are robust across regimes, while the mechanistic baseline benefits much more strongly from the simpler fixed controller.

The regime comparison yields three useful conclusions. First, the direct-condition learned models are robust: `multitask_neural_tpp` remains the strongest condition forecaster in both settings, with mean balanced accuracy 0.8116 under fixed control and 0.8146 under adaptive control. Second, `continuous_tpp` is nearly regime-invariant on the same metric (0.7891 fixed versus 0.7880 adaptive), which suggests its timing-aware inductive bias transfers cleanly across controller behavior. Third, `mechanistic` changes much more sharply with regime: its one-step time MAE improves from 2.1114 under adaptive control to 0.6006 under fixed control, which confirms that hand-structured controller assumptions help most when the controller itself is simpler and more regular.

Table 2 makes the regime comparison more specific by breaking mean condition balanced accuracy out by condition class for representative models. Each cell reports `fixed/adaptive` at the same 60/20 benchmark scale.

Condition	Mechanistic	MT Neural	Continuous	Transformer
Congested	0.722 / 0.742	0.764 / 0.782	0.703 / 0.752	0.754 / 0.800
Severe queue	0.707 / 0.738	0.806 / 0.806	0.795 / 0.730	0.768 / 0.798
NS pressure	0.589 / 0.584	0.873 / 0.870	0.871 / 0.879	0.852 / 0.812
EW pressure	0.577 / 0.603	0.834 / 0.870	0.816 / 0.857	0.799 / 0.858
Imbalance	0.557 / 0.504	0.780 / 0.745	0.761 / 0.723	0.750 / 0.738

Table 2: Per-condition mean balanced accuracy under fixed and adaptive control for representative models at the same 60/20 scale. Each cell shows fixed/adaptive performance, not a mixed-scale aggregate.

This per-condition view shows that the regime effect is not uniform. Adaptive control helps the learned models most on coarse congestion and east-west pressure conditions, where controller responsiveness changes the future queue regime. Fixed control remains relatively favorable for imbalance-oriented forecasts, especially for the mechanistic model, because the simpler controller makes directional pressure asymmetries more regular and easier to anticipate from hand-crafted logic.

7 Interpretation

Three patterns matter most:

1. **Event prediction and condition prediction are now clearly different tasks.** Strong next-event models are not automatically strong condition forecasters.
2. **Direct condition heads are worth the complexity.** They are now clearly better than deriving conditions only from autoregressive event rollout.
3. **The strongest models win for understandable architectural reasons.** Their inductive biases line up with the actual structure of the benchmark.

The strongest model in the report for condition forecasting is `multitask_neural_tpp`. It works well for three reasons. First, its shared recurrent encoder is strong enough to summarize the local event process, so it does not lose the sequential information needed to understand whether the system is building toward arrivals, departures, or a phase transition. Second, it does not stop at next-event prediction: it has direct heads trained specifically for the $10s$, $30s$, and $60s$ condition tasks. That means the representation is optimized for the exact forecasting question being asked in evaluation. Third, the model uses calibrated probability outputs, so its good balanced accuracy is matched by genuinely strong Brier and log-loss values rather than only favorable thresholded decisions.

`continuous_tpp` works especially well on timing because its hidden state evolves as a function of elapsed time, not just event count. That matters in this simulator because arrivals and departures are irregularly spaced, and queue state can change meaningfully as time passes even if no new event has yet been observed. The continuous-time decay gives the model a more natural way to represent asynchronous event gaps, which is why it remains the best one-step timing model while also staying competitive on condition forecasts once direct condition heads are added.

`transformer_tpp` works well for a different reason: it can directly re-attend to multiple earlier events rather than compressing everything into a purely recurrent state. In this application, the next condition is often determined by a pattern across several recent arrivals, departures, and phase changes, not only by the last one or two events. The transformer therefore benefits from broader context access, especially when queue pressure is building over several alternating events. Earlier in the project it lagged the best multitask GRU clearly, but the richer benchmark changed that. Once its condition path was separated from the event trunk and the condition-loss balance was tuned more carefully, it became effectively tied with `multitask_neural_tpp` on richer condition forecasting while remaining competitive on event prediction.

`neural_tpp` without direct condition heads is an instructive contrast. It is a good next-event model, but a weaker condition model, because it must answer condition questions indirectly by rolling out its own predicted future events. Any small event-level errors then accumulate before the final condition is computed. This explains why it can look respectable on one-step event metrics

while remaining much worse on Brier score and log loss for condition forecasts.

Condition calibration is therefore not a cosmetic addition to the benchmark. It changes the interpretation of what a "good" forecast means. A high balanced accuracy model that is badly calibrated may still be risky in downstream use, because its confidence scores cannot be trusted. By contrast, the strongest direct-condition models in this repository are not only more accurate after thresholding; they are also much better calibrated. That is why they improve simultaneously on balanced accuracy, Brier score, and log loss. The practical meaning is that they are better not only at deciding whether a condition will happen, but also at expressing how certain they are about that decision.

`mechanistic` performs better than the simplest empirical baselines because it explicitly encodes queue and controller structure. It knows that departures depend on service availability and that phase changes follow a constrained controller policy. That structural prior helps it remain competitive on some condition-style quantities even though it lacks the flexibility of the learned direct-condition models. Its weakness is that it cannot adapt as well to the full stochastic variability present in the simulator, especially when the effective future depends on subtle combinations of arrival bursts and state history.

The new fixed-versus-adaptive 60/20 comparison makes the model behavior more interpretable. Under fixed control, `mechanistic` becomes much more effective because its structural assumptions are closer to the true generator. Under adaptive control, those same assumptions become less reliable, and the learned models gain relative advantage. This is exactly the kind of pattern one would hope to see in a credible benchmark: hand-crafted structure helps when the environment is structurally aligned with it, while learned models become more valuable as controller behavior becomes more state-dependent.

The new baseline-versus-richer comparison adds a second important robustness check. It shows that the current ranking is not purely an artifact of one very simple environment. The richer simulator changes the leaderboard quantitatively without collapsing it qualitatively: the weaker empirical models degrade more sharply, `continuous_tpp` loses headroom on condition forecasting, and the two strongest direct-condition models — `multitask_neural_tpp` and `transformer_tpp` — emerge as the most robust richer-simulator specialists. That is exactly the kind of pattern one would hope to see if the richer simulator is increasing structural difficulty rather than merely rescaling all results uniformly.

8 Limitations and Next Steps

Important limitations remain:

- only one application domain is implemented
- the simulator is still intentionally simple
- the learned models are supervised predictors rather than full likelihood-based temporal point processes

The strongest next step is:

1. deepen the same-scale fixed-versus-adaptive comparison with per-condition calibration analysis
2. continue improving the strongest direct-condition models before adding another new archi-

tecture family
3. continue expanding the richer simulator with turning movements, spillback-style constraints, and less stationary demand regimes

9 Conclusion

The repository now supports a substantially more credible benchmark than the initial version. The main conclusions are:

- the traffic simulator is now a useful testbed for both event and condition forecasting
- direct-condition heads are a major improvement over event-rollout-derived condition forecasts
- larger train/eval sets materially improved result quality
- `multitask_neural_tpp` remains the strongest overall condition forecaster in the main benchmark tables, but `transformer_tpp` is now effectively tied with it on the richer adaptive benchmark after the latest condition-path refinement
- `continuous_tpp` remains strongest on timing
- `transformer_tpp` improved enough to become a top-tier richer-simulator condition model rather than merely a secondary contender
- `mechanistic` is far more competitive under fixed control than adaptive control, which validates the regime sensitivity of the benchmark
- the richer simulator makes event forecasting materially harder while preserving the main qualitative advantage of direct-condition learned models